



SmartExam

AI-Driven Lab Exam Monitoring and Management System

FYP-1 Deliverable — Modules, Features & Technology Stack

Zain Ali Mehmood (2280133) · Muhammad Haider Ali (2280354) · Roshaan Ali Shah (2280327)

Supervisor: Ms. Zainab Iftikhar — SZABIST Islamabad

Document 1 of 3 · Generated July 2026 · Covers the six FYP-1 modules (thesis §1.7.3, Sprints 1–4 scope)

System Architecture Overview

SmartExam is a three-tier distributed system. A locked-down **WPF desktop client** runs on each lab workstation, a **React web panel** serves teachers and administrators, and an **ASP.NET Core REST API + SignalR hub** coordinates everything over a cloud **PostgreSQL** database.

TIER	TECHNOLOGY	ROLE
Student Client	WPF, .NET 9, C#/XAML, WMI (System.Management), Win32 P/Invoke (user32.dll)	Secure exam console: HWID-bound login, exam dashboard, pre-exam checks, heartbeat telemetry, focus-loss violation detection
Web Panel	React 19, TypeScript, Vite, Tailwind CSS, Axios, @microsoft/signalr, React Router 7	Admin console (users, labs, bindings, audit) + Teacher console (exam authoring, seating, eligibility, live proctoring, results)
API Server	ASP.NET Core 9 Web API, SignalR (WebSockets), JWT Bearer auth, EF Core 9 + Npgsql, QuestPDF, MailKit	REST endpoints, role-based authorization, real-time telemetry hub, audit trail, grading pipeline
Database	Neon Serverless PostgreSQL (cloud), EF Core migrations, 22 relational tables (3NF)	Users, bindings, labs/workstations, exams/questions/test-cases, sessions/answers, monitoring events, grading, plagiarism, audit logs, notifications

FYP-1 Module Breakdown

Per the thesis sprint plan, FYP-1 delivers the foundational half of the 12-module system. Each module below is **fully implemented, integration-tested, and demo-ready**.

Module 1 — Authentication & Device Binding

FEATURES DELIVERED

- Email/username + password login for all roles; per-role redirects (Admin / Teacher / Student portals)
- JWT token authentication (signed HS256, 8-hour expiry, sub/jti/role/name/email claims)
- **Hardware fingerprinting** on the student client: CPU ProcessorId + motherboard serial (via WMI) + primary MAC address
- Fingerprint **SHA-256 hashed server-side** before storage — raw device IDs never stored
- **First-login-binds**: the first machine a student logs in from becomes their bound device (one-to-one, DB-enforced unique)
- Every later login **compares HWID server-side**; mismatch → login blocked (401 DEVICE_MISMATCH) + audit record
- **Machine identity capture**: the client reports the Windows computer name and Windows account (e.g. LAB-PC-07 / ali.hassan) so staff can see who is on which physical PC
- Admin can reset a binding; student re-binds on next login
- **Session revocation enforced**: logout, admin force-logout, and account deactivation invalidate live tokens immediately (per-request JTI + IsActive check)
- Change-password (verifies current password, min 8 chars) via the web panel profile menu
- **Security audit trail**: LOGIN_SUCCESS / LOGIN_FAILED / DEVICE_BOUND / DEVICE_MISMATCH / LOGOUT / PASSWORD_CHANGED all logged with actor, timestamp, details

TECH STACK

ASP.NET Core JWT Bearer

SHA-256 (System.Security.Cryptography)

WMI Win32_Processor / Win32_BaseBoard

System.Net.NetworkInformation (MAC)

EF Core unique index (UserId)

OnTokenValidated middleware

Module 7 — User & Role Management

FEATURES DELIVERED

- Five roles: SuperAdmin, Admin, Teacher, **Invigilator**, Student — strict separation enforced by `[Authorize(Roles=...)]` on every endpoint
- Create user (role + initial password), **edit user** (name / email / role / optional password reset), **delete user**
- Delete guards: cannot delete yourself; cannot delete a teacher who owns sections; students with exam history must be deactivated instead (preserves records)
- **Deactivate / reactivate** accounts — deactivation also revokes live sessions instantly
- **Batch CSV import** (name, email, password, role) with per-row duplicate/invalid reporting
- Device-binding management page: binding status, **bound PC name**, **Windows user**, **last-seen time**, reset + force-logout actions
- Role-filtered user list with search; all admin actions written to the audit log

TECH STACK

ASP.NET Core REST (UsersController)

Salted SHA-256 password hashing

React slide-over forms

Client-side CSV parsing (FileReader)

AuditService

Module 8 — Exam Creation & Configuration

FEATURES DELIVERED

- 4-step exam wizard: Basic Info → Questions → Rules & Settings → Assign Students
- Exam metadata: title, **instructions** (shown on the student client), course/section, scheduled date & start time, duration
- Three question types: **Coding** (with public + hidden test cases), **Theory**, and **MCQ** (options + correct-answer key)
- Optional **reference answer / sample solution** per Coding/Theory question — never exposed to students
- Allowed-applications whitelist (exe names) delivered to the student client; plagiarism threshold slider; AI-evaluation toggle
- **Edit exam** (Scheduled only — refused with 409 once Active/Ended) with a fully pre-filled wizard; **delete exam** (same guard)
- Server-side validation: ≥ 1 question, marks > 0 , MCQ needs ≥ 2 non-empty options + valid key, coding needs ≥ 1 test case, threshold 0–100
- MCQ auto-grading on submission (exact match → full marks / 0); MCQs excluded from plagiarism comparison
- Students never receive hidden test cases, correct-option keys, or reference answers (server strips them)

TECH STACK

EF Core owned graphs (Exam→Question→TestCase)

React multi-step wizard state

JSON options storage (OptionsJson)

enum-as-string persistence

Module 9 — Eligibility & Access Control (Seating)

FEATURES DELIVERED

- **Lab & workstation inventory**: admins register labs and seats (row/column naming, e.g. R1C1) with per-seat IP addresses
- **Seating map**: teachers assign each student a specific seat from a lab dropdown on the Eligibility page
- **Seat-conflict detection**: assigning an already-taken seat in the same exam is rejected (409) — thesis UCT-04 exception flow
- Per-student **eligibility toggle + reason note**, mark-all controls, batch save; every eligibility decision audited (thesis FR9)
- **Add a student to an existing exam** (enrolled-but-unassigned dropdown) and **remove a student** (blocked if they already have an exam session)
- Student's lab, seat and proctor name shown on their desktop dashboard

TECH STACK

LabsController + ExamAssignments REST

EF Core FK (Assignment→Workstation)

React controlled selects w/ optimistic reload

AuditService (WORKSTATION_ASSIGNED / STUDENT_ASSIGNED ...)

Module 3 — Student Exam Dashboard (Desktop Client)

FEATURES DELIVERED

- Redesigned secure exam console (borderless custom chrome, minimize/close, draggable title bar)
- Shows the student's **assigned exam** (nearest-open exam selected automatically), status, proctor, teacher instructions, and **permitted-application chips**
- **Live countdown timer** — server-synchronized every heartbeat; turns red under 5 minutes; picks up teacher time-extensions automatically
- **Pre-exam checklist**: Attendance check-in + Identity verification with green tick states — both required before Start Exam unlocks
- Lab / Seat / Monitoring status rail; live connection pill (green/red)
- **Proctor Messages panel**: polls teacher warnings & announcements every 15 s with an unread badge
- **Session-kick protection**: if an admin deactivates the account or forces logout, the client detects it within seconds and returns to the login screen

TECH STACK

WPF MVVM (INotifyPropertyChanged)

DispatcherTimer (countdown + session watch)

Custom ControlTemplates (App.xaml)

DataTriggers for state visuals

HttpClient typed ApiClient

Module 10 — Live Proctoring Dashboard (+ Module 5 telemetry layer)

FEATURES DELIVERED

- **Real-time grid** of active students over SignalR WebSockets — one card per student with green / orange / red status
- Per card: seat, **real PC name + Windows account**, active window title, process-list snapshot, answered/total progress bar, violation count
- Client heartbeats every 10 s (foreground-app flag, active window, top-12 process list) — captured via Win32
`GetForegroundWindow` / `GetWindowText`
- **Focus-loss violation detection**: switching away from the exam window raises a Violation (transition-based, 30 s cooldown) → card turns red + timeline entry
- Per-student **activity timeline drawer**; teacher can **send a warning message** (arrives in the student's Proctor Messages panel)
- **Force-submit one student, force-end exam for all**, and **Extend Time** (1–180 min) — students' countdowns resync on the next heartbeat
- Security: telemetry broadcast only to the proctors of that exam (SignalR groups); hub actions role-checked; teachers can only act on their own sections' sessions

TECH STACK

SignalR hub + groups (exam:{id})

@microsoft/signalr client

Win32 P/Invoke telemetry

System.Diagnostics.Process

React Map-based live state

Cross-Cutting Capabilities

- **Audit trail (foundation of Module 12)**: every security-relevant event — logins, device decisions, user admin, eligibility, seating, time extensions — is written to an immutable AuditLog table and viewable at Admin → Audit Logs.
- **Post-exam pipeline (foundation of Modules 11/13)**: auto-save answers, submission, AI-simulated grading with teacher overrides, pairwise plagiarism similarity with threshold flagging, PDF report generation (QuestPDF), and exam analytics dashboards.

- **Demo data management:** the database seeds only when empty; admins can reset to a clean labelled "(Demo)" dataset via `POST /api/admin/reset-demo-data`.

Deferred to FYP-2 (per thesis sprint plan §1.7.3): Module 2 (pre-exam hardware verification & mock test, camera-based identity), Module 4 (fullscreen lockdown, keyboard hooks, Task-Manager/USB blocking), Module 5 full capture (periodic screenshots & webcam), Module 6 (encrypted local backup & crash recovery), Module 11 real AI (Docker code execution, sentence-transformer semantic grading), Module 12 full reporting suite, IP-based access enforcement, and refresh-token rotation.